Transactions App Architecture

Accurate package-level diagram with app wiring, reusable views, contracts, persistence, and remote data flow

TransactionsApp

App composition and feature wiring

Key Objects

- TransactionsApp
- RootView
- AppContainer
- SBTransactionManager
- SBTransactionListView
- SBTransactionDetailView

Responsibilities

- Creates the app entry point and root navigation
- Builds dependencies in AppContainer
- Connects manager and views for the Transactions feature

hosts feature screens

SBViews

Reusable SwiftUI UI package

Key Objects

- SBCardView
- SBTransactionRowView
- SBLabelValueSection
- SBPrimaryButton
- SBSectionDivider
- SBSpacing
- SBSize
- SBRadius

Responsibilities

- Provides shared visual components and layout primitives
- Keeps feature screens consistent and reusable
- Includes previews for fast UI iteration

injects repository implementation

SBProcess

Repository implementation package

Key Objects

- SBTransactionRepositoryImpl

Responsibilities

- Implements SBTransactionRepository
- Coordinates local and remote data sources
- Acts as the feature orchestration layer

implements contract

SBDomain

Pure contracts and business models

Key Objects

- SBTransaction
- SBMoney
- SBTransactionType
- SBTransactionType+Extensions
- SBTransaction+Extensions
- SBTransactionRepository
- SBTransactionLocalRepository
- SBTra

Responsibilities

- Defines shared models and enums
- Defines repository and service contracts
- Remains independent from storage, network, and UI

defines local contract

uses local repository

uses remote service

SBLocal

Local persistence package

Key Objects

- SBTransactionEntity
- SBTransactionRecord
- SBTransaction+Entity
- SBTransactionEntity+Domain
- SBTransactionEntity+Record
- SBTransactionRecord+Entity
- SBInMemoryTransactionLocalRepository
- SBSwiftDataTransactionLocalRepository

Responsibilities

- transaction data
- Maps between domain, entity, and record layers
- Supports both in-memory and SwiftData implementations

defines remote contract

SBNetwork

Remote DTO and service package

Key Objects

- SBTransactionDTO
- SBTransactionListDTO
- SBTransactionAmountDTO
- SBTransactionTypeDTO
- SBTransactionDTO+Mapper
- SBTransactionListDTO+Mapper
- SBTransactionAmountDTO+Mapper
- SBTransactionTypeDTO+Mapper
- SBJSONResourceLoader
- SBStubTransactionRemoteService

Responsibilities

- Loads transaction data from resource-backed services
- Maps DTOs into domain transactions
- Supports testable remote behavior with stubs

Primary Runtime Flow

1. TransactionsApp creates AppContainer and wires the Transactions feature.
2. SBTransactionListView and SBTransactionDetailView use reusable building blocks from SBViews.
3. The feature talks to SBTransactionRepository through SBTransactionRepositoryImpl in SBProcess.
4. SBTransactionRepositoryImpl uses SBDomain contracts to read local data from SBLocal and fetch remote data from SBNetwork.
5. SBLocal persists transactions through SBInMemoryTransactionLocalRepository or SBSwiftDataTransactionLocalRepository.
6. SBNetwork loads DTO data through SBJSONResourceLoader and exposes it via SBStubTransactionRemoteService.